SWEN30006 Software Modelling and Design
Workshop 7: GoF Patterns (Part 1)
Facade, Decorator and Builder

School of Computing and Information Systems
University of Melbourne
Semester 1, 2025

Completion: You *should* complete the exercises ***as a team***. To receive a workshop mark, you **need** to demonstrate your active participation during the workshop. Your tutor will observe your participation for you to be eligible to receive a mark for this workshop. The active participation include (but not limit to) present your solutions, help your teammates, and actively engage with the team. Attending the workshop with only passive participation will *not* be given a mark. See the Workshop Overview under Subject Information on the LMS for more details.

Requisite Knowledge and Tools

To complete this workshop you will need to be familiar with the following terms and concepts:

- Class Diagram
- GRASP principles
- GoF Facade, Decorator and Builder.

We assume that you have reviewed the **Pre-Workshop Learning** in Workshop 7 (Week 8) module in LMS when commencing this workshop activity.

Workshop Objectives

In this workshop, we will be focusing on applying three GoF patterns, **Facade**, **Decorator**, and **Builder**, to a particular problem. The exercises begin with designing a system using provided information, and then analysing how the GoF patterns can be used to solve design problems. Finally, we will finish by implementing the system. By the end of this workshop, you should:

- Be able to apply GoF patterns to solve design problems.
- Be able to create appropriate models to represent the design.
- Be able to implement the system based on the GoF patterns.

Introduction

Below is a brief summary of three GoF patterns that we will be using in this week's exercises.

- **Facade:** The Facade pattern is a structural pattern. This pattern is often used in real-world applications. The Facade pattern is ideal when working with a large number of interdependent classes which are complicated to use. Hence, the facade pattern is used to define a simplified interface of such a complex subsystem. By using the Facade pattern, the client class that uses this complex system does not need to know all the classes and methods of this complex subsystem. The client class just can simply call few functions from the facade object.
- **Decorator:** This is a structural pattern that allows users to add new responsibilities to an existing object without modifying its structure or subclassing. For instance, adding a class to decorate the border of a given rectangle (shape) without changing how the rectangle is currently being drawn.
- **Builder:** Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code. For instance, consider the construction of a home. Home is the final end product (object) that is to be returned as the output of the construction process. It will have many steps like basement construction, wall construction, and so on roof construction. Finally, the whole home object is returned. Using Builder pattern, you can build homes with different properties

The System Under Discussion: (Regular) Monopoly

This week you will be *extending* the Larman's Case Study system (i.e., a simulation of Monopoly). Figure 1 shows a partial domain model of the current Monopoly system. See partial design models from the textbook in the separate file **Monopoly_DesignModel.pdf**.

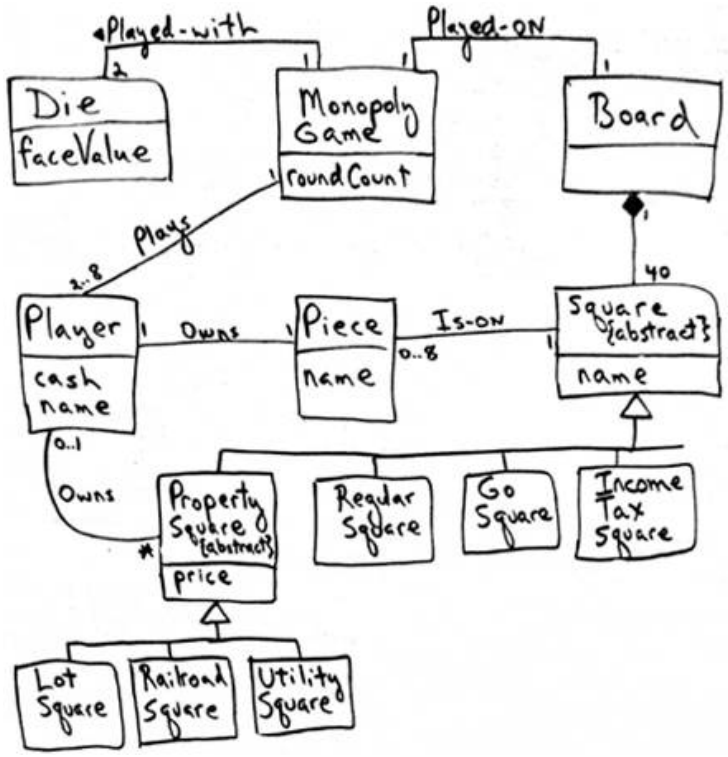

Figure 1: Domain Model of Monopoly

A brief description of the current system is as follow. The system gets the number of players as an input from the user and initialises the board and the players. Then, the system runs for 20 rounds and stops. In this system, the board has 40 squares:

- 1 × Go square
- 1 × IncomeTax square
- 1 × Jail square
- 3 × Go to Jail squares
- 3 × Rail Road squares
- 2 × Utility squares
- 3 × Lot squares
- 26 × Regular squares

The current rules for these squares are:

- If a player lands on a **Rail Road, Utility, or Lot** square, the player will attempt to purchase the square. If they successfully purchase a square, other players have to pay the rent to the owner when they land on that square.
- If a player lands on the **Go to Jail** square, the player has to move to the **Jail** square.
- If a player lands on the **IncomeTax** square, the player has to pay 10% of their net worth (or $200 at maximum) as tax.
- If a player lands on the **Go** square, the player gets $200 cash.
- If a player lands on the **Jail** or **Regular** squares, the system does nothing.

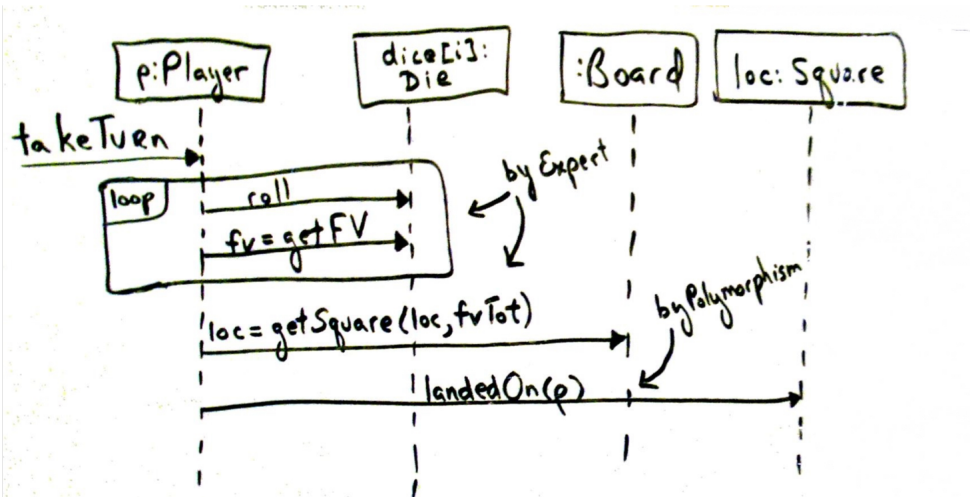

Figure 2: System Sequence Diagram: Dynamic Behaviour when a player takes turn.

Part 1 Adding LuckCards

Now, you will be adding one more feature – LuckCards – to this system. LuckCards are used only when a player lands on the **Go to Jail** square. The player will randomly draw 1 card from the card deck and follow the instruction described in the card. There are 5 cards in the deck:

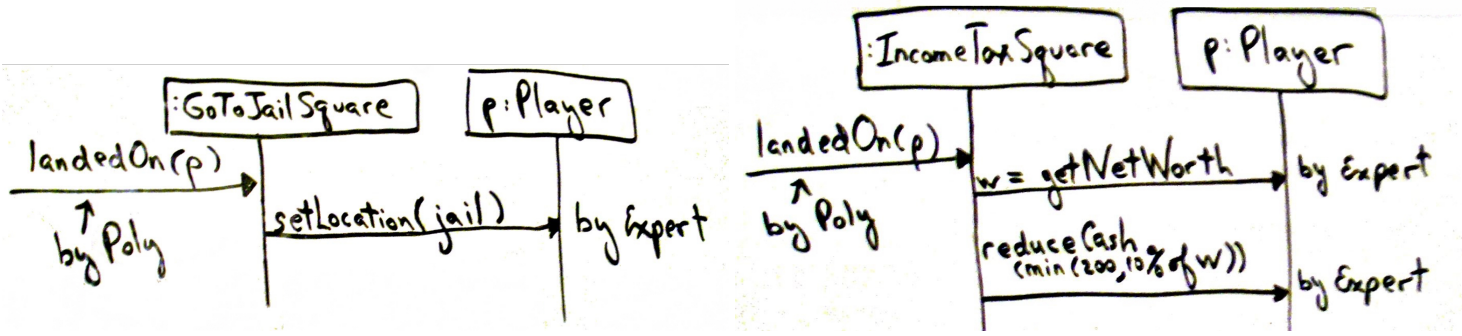

Figure 3: System Sequence Diagram. (Left) Go to Jail case, (Right) Income Tax case

- 1 × JailExcemptionCard: If the player gets this card, the player stays on the Go to Jail square. No need to move the Jail square.
- 2 × GoToJailCard: If the player gets this card, the player moves to the Jail square.
- 2 × PayJailFeeCard: If the player get this card, the player pays $500 to exempt from Jail. If the player's cash is not sufficient, the player has to move to the Jail square.

Requirement: If you look at the SSD in the figures above, you should be able to identify that this feature should be added into the `GotoJailSquare` class. However, when adding this feature, the current GotoJailSquare class should have very **low impact**. Ideally, only one statement should be added into the existing class of the Monopoly system. Based on this low impact requirement, the **Facade** pattern should be used to implement this feature. Also consider other applicable patterns.

Q1.1 Modelling

Using the description above, create a design class diagram for classes to show how can we use the Facade pattern for this additional feature.

Q1.2 Implementation

Using your design from the previous sub-task, implement this feature into the current Monopoly system. A Java package for Monopoly is provided in GitHub assignment: https://classroom.github.com/a/xtJJs-un
Note that this GitHub assignment has GitHub Actions for automated build (but no automated tests).

Part 2 Building a House

In the original behaviour, nothing happens when a player lands on a Regular square. So, let's spice things up! Now, when a player lands on an empty Regular square (i.e., a Regular square without a house), that player can build **a house** and own the square with a house. After that, if the owner (i.e., the same player who built a house) lands on his/her regular square with a house, the owner builds **a garden**.

A regular square which has a house (and a garden) generates income to the owner, which is collected every time as the owner lands on that square. Income is generated at the following rate:

- A Regular square with a house generates $20 to the owner
- If a Regular square has a garden, it also generates ($5×#Visits) to the owner
  Note: The counter of visits starts when a garden is built and it counts every time when any player lands on that sqaure.

```
Player 6: dice total = 8 move to Square 16 with a house and a garden
Square 16 with a house generates 20 of income to Player 6
Square 16 with a house and a garden generates income of 25 to Player 6
```

Figure 4: An example of log for a regular square with a house and a garden, where Player 6 is the owner of Square 16 and Square 16 with a house and a garden has been visited 5 times by any player (i.e., $5×5=$25

Requirement: Instead of modifying the existing code for Regular square to use an if-else statement, we would like to **modify its behaviour at runtime**. Hence, the **Decorator** pattern should be used to implement this feature.

Q2.1 Modelling

Using the description above, extend your a design class diagram for classes to show how can we use the Decorator pattern for this additional feature.

Q2.2 Implementation

Using your design from the previous sub-task, implement this feature into the current Monopoly system.

Minimum Expectation: The system is already complex. It is acceptable if you cannot follow some of the GRASP principles. At least, you should try to apply the Decorator pattern.

Part 3 Building different kinds of house (Extra)

As an extension of the monopoly game, it happens that a player could earn more by building different types of houses when they land on a regular box. A player can build either a single-pen, a saddlebag, or a dogtrot house. The type of house a player is allowed to build is determined by the player's current net worth while landing on a regular box: $50-$100 can build a single-pen; $101-$300 can build a saddle bag while $301 and above can build a dogtrot house. Each house is generally built by stacking a foundation, a given number of rooms and a roof. However, building the foundations, rooms and roofs for each type of house requires complex domain knowledge.

The player doesn't want to know anything about the complexities of constructing any of the types of houses and therefore outsources the house construction. They simply provide their net worth and wait for their house so that they can know the amount to charge whenever a visitor gets to their houses.

The construction company has provided a jar file (regularHouseBuilder.jar) that has the domain knowledge to build each type of house when a player provides the network. They expect you to use the Builder pattern to integrate this API into their existing monopoly game. The class diagram for the jar file is given in figure 3.

Q3.1 Modelling

Draw a Sequence diagram showing how you can calculate the amount generated per visitor for a player whose net worth is $175.

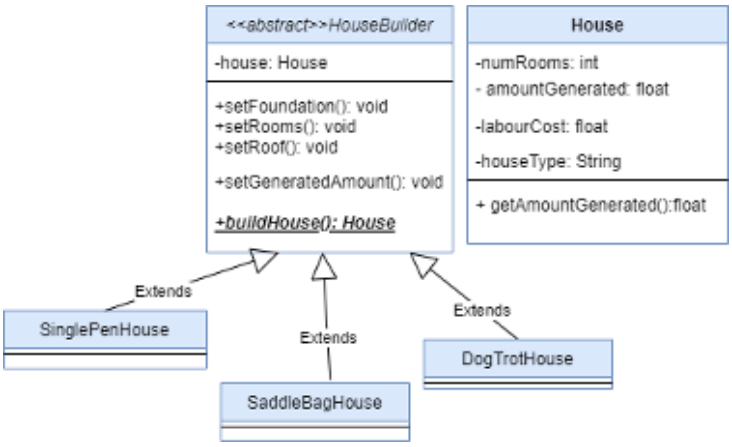

Figure 5: Partial Class diagram for RegularHouseBuilder. The classes are contained in a com.builder package.

Q3.2 HouseManager Implementation

Write a HouseManager director that builds the appropriate type of house depending on the net worth of a player.

Q3.3 Test Program

Either by adding a code to the RegularSquareBox or by writing an independent client program, output the amount generated per visitor for players whose net worth are $75, $290 and $610 respectively.